调用更少更轻的画布API

https://codepen.io/huanjinliu/embed/OJaQJbQ?editors=1010

1）为什么一次 `fill` 会更快呢？

正常人想那不是当然的吗，事情少做了当然快了，是的没错。

那少做了什么呢，有人会觉得因为是页面中画布绘制的次数更少，是这样吗？

首先这里提到了画布绘制，我们先得了解一件事：画布的内容更新会造成页面重绘吗？

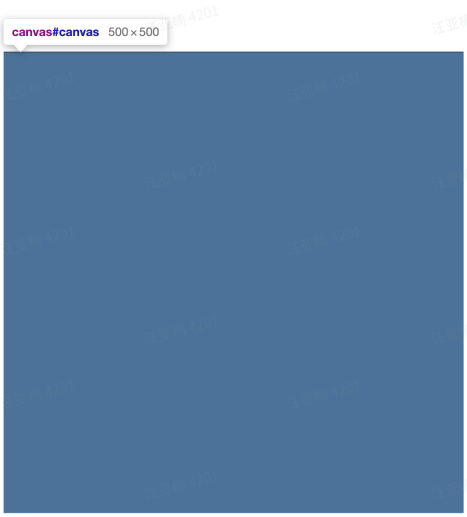
解答：是会的，如上述程序中画布更新，chrome开发面板中我们是能找到绘制的节点的：

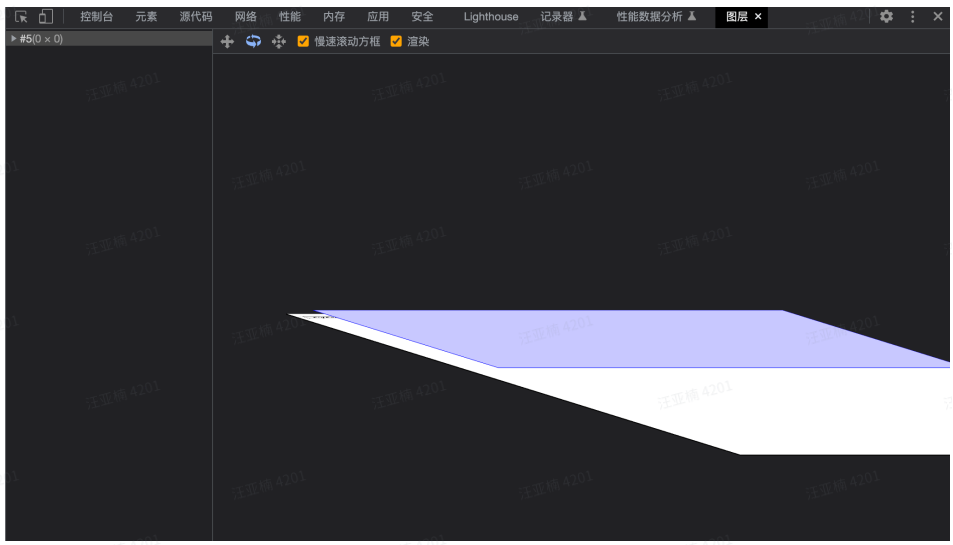
那会引起重排吗？

不会，因为元素布局没有发生改变。

画布的重绘是整个页面的重绘吗？

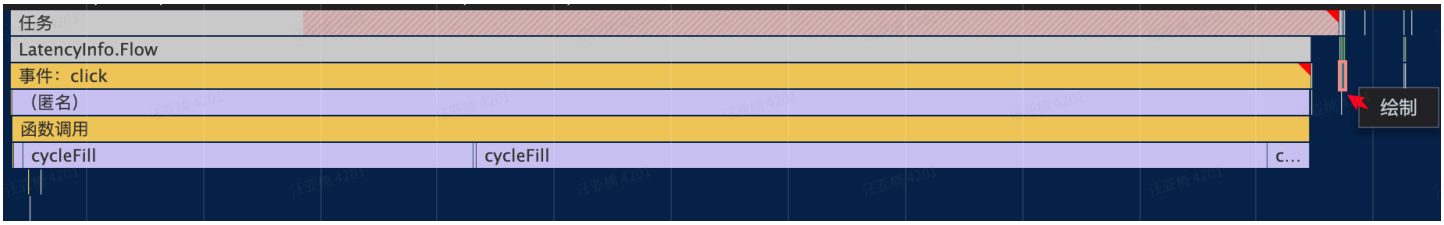
不是，画布的创建是会自动处于一个独立的图层，而画布的更新绘制也只是所在图层重新绘制。



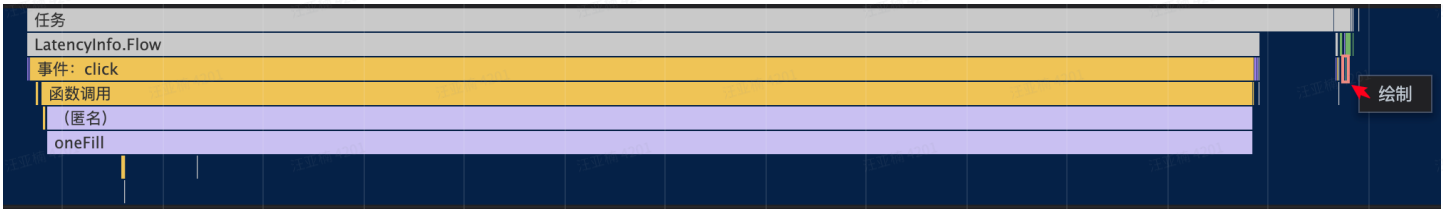


既然会重绘，那我们来看看 `performance` 面板是否真的如我们所想出现了多次重新绘制：

CircleFill



OneFill

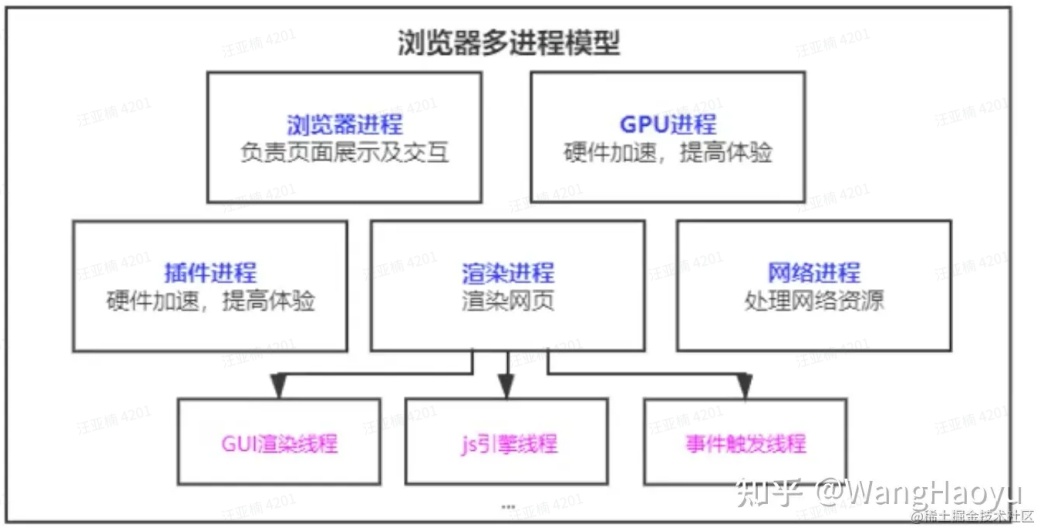


我们会很惊奇发现，无论是循环还是一次绘制实际上都只会在循环后执行一次绘制。

原因

因为JS引擎线程长时间执行，导致GUI渲染线程被挂起，只有等到JS线程执行完成后才立即执行。而等到JS线程执行完成后，画布的像素值也都统统计算好放在缓冲内了，这时理所当然也就只绘制一次就好了。

用于渲染的GUI线程和脚本运行的JS引擎线程都处在浏览器的渲染进程中，它们的运行是互斥的。



如果你就是要看到循环中的每一帧画布变化，你只要把绘制包在 `setTimeout(..., 0)` 中即可实现，但正常讲没有这个必要。

2） `fillRect` 比一次 `fill` 慢的真正原因

从 `blink` 源码中看：

```
// 关键代码如下
void BaseRenderingContext2D::fillRect(double x, double y, double width, double height) {
        ...
  ValidateRectForCanvas(x, y, width, height);  // 检验参数
  rect(x, y, width, height);                   // 创建矩形路径对象
  Draw();                                      // 绘制矩形
  ...
}
```

`fillRect` 的源码实现不过也就是帮我们将 `rect` 和 `fill` 打包起来用，除此它还多了一些如：检验参数的操作，而这些才是真正导致它在循环中调用慢于一次 `fill` 的原因。

所以这个示例本身想强调的是使用更少更轻的画布 `API` 来实现优化。

3）消耗较大的画布API

如何调用更少的画布 `API` 以实现想要的功能是我们在后续 `canvas` 使用中需要不断学习和注意的，以下画布API需重点留意（括号内为性能消耗大的原因）：

1. 阴影 Shadows 中的 shadowBlur （大量的像素值计算）
2. ctx.strokeStyle、ctx.fillStyle（需要解析颜色字符串）
3. ctx.save、ctx.restore（栈的维护及状态恢复）

思考：还记得这个问题吗，它除了效果问题还有性能问题吗？

> 为什么不简单快捷为每个小实体格子都绘制一个矩形，以此组合成最终形状？

解答：如果循环内多次调用绘制确实会比一次调用的性能差些，但是因为实际游戏中铺设方块的数量是比较少的，所以其实两者所造成的性能差异是可以忽略不计的，但是咱们在编码中还是需要多注意循环可能造成的代码性能问题。

循环中多次绘制：

```
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');

const block = [[0, 1, 0], [1, 1, 1], [0, 1, 0]];
const size = 33.33;

for (let row = 0; row <= 2; row++) {
  for (let col = 0; col <= 2; col++) {
    if (block[row][col] === 0) continue;
    // 每遍历一个格子都调用绘制
    ctx.fillRect(row * size, col * size, size, size);
```

```
    }
  }
```

一次调用：

```
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');

const block = [[0, 1, 0], [1, 1, 1], [0, 1, 0]];
const size = 33.33;

// 清空旧的路径
ctx.beginPath();
for (let row = 0; row <= 2; row++) {
  for (let col = 0; col <= 2; col++) {
    if (block[row][col] === 0) continue;
    // 每遍历一个格子只是添加了矩形路径到路径中
    ctx.rect(row * size, col * size, size, size);
  }
}
// 只调用一次绘制
ctx.fill();
```

参考

1. HTML 渲染那些事儿